

AWS Cloud Security and Identity Management And Automated Security and Case Studies

The Critical Importance of Cloud Security

As organizations rapidly migrate to cloud-native environments, security has become the foundational pillar of successful cloud adoption. AWS processes billions of API calls daily, making robust security controls not just important—but essential.

Cloud security differs fundamentally from traditional on-premises security. The distributed nature of cloud resources, dynamic scaling, and shared infrastructure models create unique challenges that require new approaches and mindsets.

Why This Matters

- Cloud breaches cost organizations an average of $4.45 million per incident
- 95% of cloud security failures are due to customer misconfiguration
- Identity and access management issues cause 80% of data breaches
- Automated security monitoring reduces incident response time by 73%

AWS Security Philosophy: Security by Design



Defense in Depth

Multiple layers of security controls protect resources at every level—from physical data centers to application code.



Automation First

Security processes are automated by default, reducing human error and enabling rapid response to threats.



Full Visibility

Comprehensive logging and monitoring provide complete visibility into all API calls, changes, and access patterns.



Shared Responsibility

Clear delineation between AWS and customer security responsibilities ensures no gaps in protection.

AWS built security into every layer of its infrastructure. Understanding these principles helps you architect secure, compliant, and resilient cloud solutions.

Learning Objectives

01

Master the Shared Responsibility Model

Understand exactly what AWS secures versus what you must secure across different service models.

02

Implement IAM Best Practices

Configure users, roles, and policies following the principle of least privilege.

03

Secure Infrastructure Resources

Apply security controls to VPCs, EC2 instances, S3 buckets, and application layers.

04

Enable Comprehensive Monitoring

Implement logging, encryption, and automated threat detection across your environment.

05

Respond to Security Incidents

Develop incident response workflows that minimize impact and enable rapid recovery.

AWS Cloud Security Model

AWS Security Approach

AWS provides a highly secure global cloud infrastructure and offers security services to customers.

Key Principles

- Security by design
- Global infrastructure protection
- Continuous monitoring and improvement
- Compliance certifications and standards

Benefits

- Reduced operational overhead
- Scalable security controls
- Built-in security services

The Shared Responsibility Model



Understanding Security in AWS

Security OF the Cloud

AWS Responsibility

AWS manages and secures the foundational infrastructure that runs all services offered in the AWS Cloud. This includes:

- Physical security of data centers (guards, cameras, access controls)
- Hardware infrastructure (servers, storage, networking equipment)
- Virtualization layer and hypervisor
- Network infrastructure and managed services
- Compliance certifications and audits

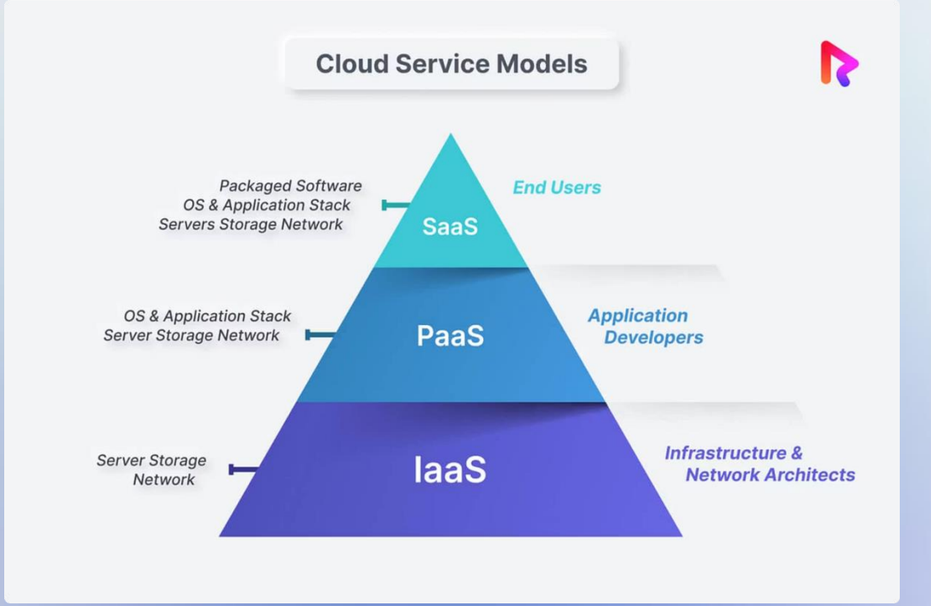
Security IN the Cloud

Customer Responsibility

Customers retain control and responsibility for security controls they choose to implement to protect their content, applications, and data:

- Data encryption (at rest and in transit)
- Identity and access management
- Operating system and application patching
- Network configuration and firewall rules
- Application code security

Shared Responsibility Across Service Models



Infrastructure as a Service (IaaS)

Example: EC2, EBS, VPC

You manage: Guest OS, applications, security groups, network ACLs, data encryption, IAM policies.

AWS manages: Physical hardware, hypervisor, network infrastructure, physical security.

Platform as a Service (PaaS)

Example: RDS, Elastic Beanstalk

You manage: Data encryption, IAM access, application code, database users and permissions.

AWS manages: OS patching, database software updates, infrastructure scaling, backups.

Serverless

Example: Lambda, DynamoDB, S3

You manage: Application code security, data encryption, IAM policies, bucket/table policies.

AWS manages: Everything else—runtime environment, capacity, patching, availability.

Real-World Responsibility Examples



Amazon RDS Database

AWS: Database engine patches, automated backups, infrastructure scaling

Customer: Database user permissions, encryption keys, VPC security groups, data classification



Amazon S3 Bucket

AWS: Durability (99.999999999%), availability, infrastructure security

Customer: Bucket policies, access control lists, encryption settings, public access blocks, versioning



EC2 Instance

AWS: Physical host security, hypervisor, network infrastructure, hardware lifecycle

Customer: OS patches, firewall rules, antivirus, application security, data backup, encryption

Key Insight: As services become more managed, AWS assumes more security responsibilities, but customers always retain responsibility for their data, access controls, and encryption.

Identity & Access Management



IAM: The Foundation of AWS Security

AWS Identity and Access Management (IAM) is a free service that controls who can access your AWS resources and what actions they can perform. IAM is the gatekeeper to your entire AWS environment—every API call goes through IAM authorization checks.

Why IAM Matters

- Enables fine-grained access control to AWS resources
- Supports identity federation with corporate directories
- Provides temporary security credentials
- Offers centralized control over permissions
- Integrates with all AWS services
- Enables compliance and audit requirements

Core Principles

Least Privilege: Grant only the permissions required to perform a task—nothing more.

Separation of Duties: No single user should have permissions to complete critical operations alone.

Defense in Depth: Use multiple types of policies to create overlapping security controls.

IAM Core Components



IAM Users

Permanent named identities representing people or applications. Each user has unique credentials (password and/or access keys) and their own set of permissions. Use for individual, long-term access needs.



IAM Groups

Collections of users that share the same permissions. Groups simplify permission management—attach policies to the group rather than individual users. Users inherit all group permissions.



IAM Roles

Temporary identities that can be assumed by trusted entities (users, services, or applications). Roles don't have permanent credentials—they provide temporary security tokens. Ideal for cross-account access and service-to-service authorization.

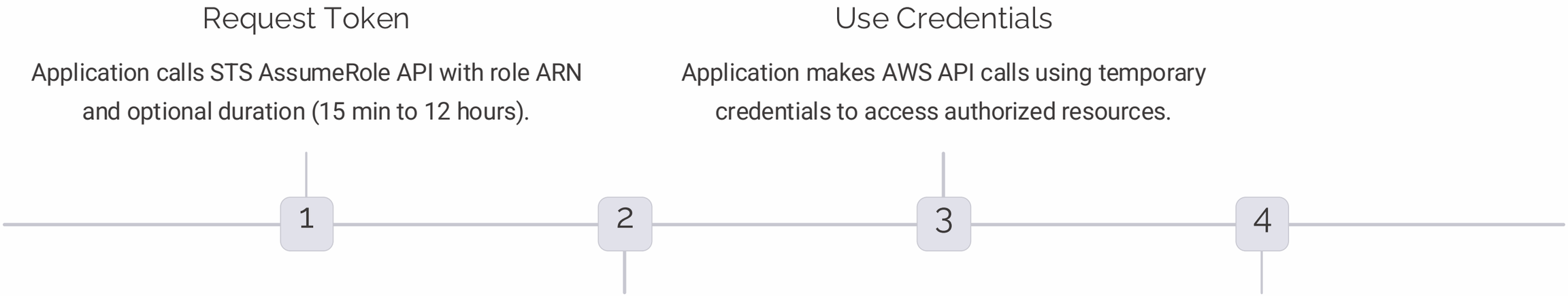


IAM Policies

JSON documents that define permissions—what actions are allowed or denied on which resources. Policies are the mechanism that enforces access control throughout AWS.

Temporary Security Credentials

AWS Security Token Service (STS) generates temporary credentials that provide time-limited access to AWS resources. This is the most secure way to grant access—credentials automatically expire, eliminating the risk of long-term credential leakage.



Types of IAM Policies



Identity-Based Policies

Attached directly to IAM users, groups, or roles. Define what actions an identity can perform on which resources. Most common policy type.

Example: Policy allowing EC2 read access



Resource-Based Policies

Attached directly to resources (S3 buckets, SQS queues, Lambda functions). Define who can access the resource and what actions they can perform.

Example: S3 bucket policy allowing cross-account access



Service Control Policies (SCPs)

Applied at the AWS Organizations level. Set maximum permissions for accounts in your organization. SCPs don't grant permissions—they restrict them.

Example: Prevent all accounts from leaving specific regions



Permission Boundaries

Advanced feature that sets the maximum permissions an identity-based policy can grant. Used to delegate permission management while maintaining control.

Example: Allow developers to create roles but limit maximum permissions

IAM Policy Structure

IAM policies are JSON documents with specific structure. Understanding policy syntax is essential for creating effective access controls.

Key Elements:

- **Effect:** Allow or Deny
- **Action:** API calls (service:Action)
- **Resource:** ARN of affected resources
- **Condition:** Optional circumstances

Policies are evaluated together. An explicit Deny always wins over Allow. If no policy grants permission, access is denied by default.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowEC2ReadOnly",
      "Effect": "Allow",
      "Action": [
        "ec2:Describe*",
        "ec2:Get*"
      ],
      "Resource": "*"
    },
    {
      "Sid": "DenyProductionTermination",
      "Effect": "Deny",
      "Action": "ec2:TerminateInstances",
      "Resource": "arn:aws:ec2:*:*:instance/*",
      "Condition": {
        "StringEquals": {
          "ec2:ResourceTag/Environment": "production"
        }
      }
    }
  ]
}
```

Least Privilege in Practice

The principle of least privilege means granting only the minimum permissions necessary to perform required tasks. This fundamental security concept dramatically reduces your attack surface and limits the damage from compromised credentials.

Start with Nothing

Begin with zero permissions. Default deny means users have no access unless explicitly granted.

Grant Specific Permissions

Add only the exact actions needed for specific resources. Avoid wildcards (*) when possible.

Use Conditions

Further restrict access based on time, IP address, MFA, tags, or other contextual factors.

Review Regularly

Use IAM Access Analyzer and Access Advisor to identify and remove unused permissions.

Refine Continuously

Iterate based on actual usage patterns. Remove permissions that haven't been used in 90 days.

IAM Best Practices

Enable MFA Everywhere

Require multi-factor authentication for all users, especially for privileged accounts. Hardware MFA devices provide strongest protection.

Don't Use Root Account

Secure root account with MFA, then create IAM users for all activities. Root account has unrestricted access—use only for account-level tasks.

Use Roles, Not Users

Applications and services should use IAM roles, not embedded access keys. Roles provide temporary credentials that rotate automatically.

Rotate Credentials

Enforce password policies requiring rotation. Rotate access keys every 90 days. Use AWS Secrets Manager for application credentials.

Monitor and Audit

Enable CloudTrail for all regions. Review IAM credential reports monthly. Use AWS Config to detect policy violations.

Centralize Identity

Use IAM Identity Center (formerly SSO) to provide single sign-on access. Federate with corporate identity provider when possible.

Identity Federation and Single Sign-On

Identity federation allows users to access AWS using existing corporate credentials from identity providers like Active Directory, Okta, or Azure AD. AWS IAM Identity Center provides a central portal for SSO access across multiple AWS accounts and applications.

Benefits of Federation

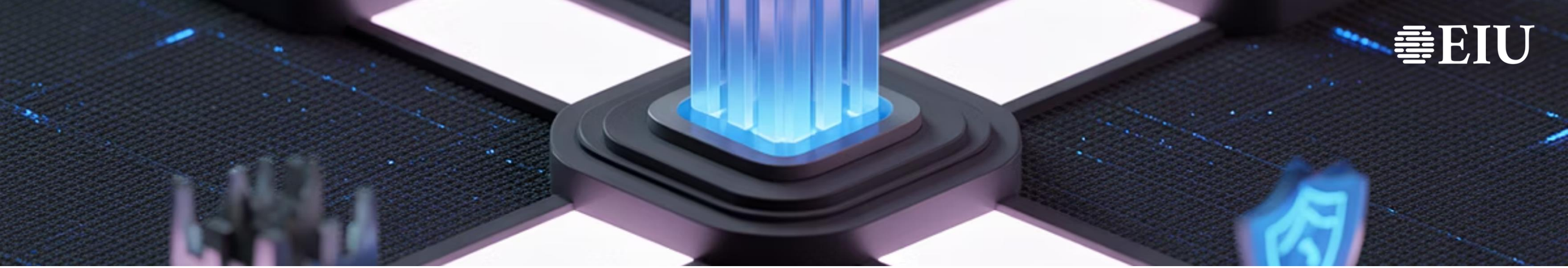
- Users authenticate once with existing credentials
- No need to manage separate AWS passwords
- Centralized user lifecycle management
- Automatic credential rotation via temporary tokens
- Enhanced security through centralized controls
- Improved user experience and productivity

Common Federation Protocols

SAML 2.0: Enterprise standard for web-based SSO. Integrates with most corporate identity providers.

OIDC: Modern protocol based on OAuth 2.0. Used for web and mobile applications.

AWS IAM Identity Center: AWS-managed solution supporting both SAML and SCIM for user provisioning.



Infrastructure Security

Amazon VPC: Your Virtual Network

Amazon Virtual Private Cloud (VPC) lets you provision a logically isolated section of the AWS Cloud. Within your VPC, you have complete control over networking—IP address ranges, subnets, route tables, and network gateways.

Subnets

Segments of your VPC's IP range. Public subnets route traffic to the internet; private subnets don't. Deploy resources based on internet access needs.

NAT Gateway

Allows private subnet resources to access the internet for updates while preventing inbound connections from internet to private resources.



Route Tables

Control traffic routing within VPC and to external networks. Each subnet associates with a route table that determines packet forwarding.

Internet Gateway

Enables communication between VPC resources and the internet. Required for public subnets to reach external services.



Security Groups vs Network ACLs

AWS provides two types of network firewalls for controlling traffic. Understanding their differences is critical for designing secure architectures.

Security Groups

Stateful firewall at instance level

- Applied to individual EC2 instances, RDS databases, load balancers
- Stateful: Return traffic automatically allowed
- Only support ALLOW rules (deny by default)
- All rules evaluated before decision
- Changes take effect immediately
- Can reference other security groups

Best for: Instance-level access control

Network ACLs

Stateless firewall at subnet level

- Applied to entire subnets
- Stateless: Must explicitly allow return traffic
- Support both ALLOW and DENY rules
- Rules processed in numerical order
- Changes take effect immediately
- Default NACL allows all traffic

Best for: Subnet-level controls, explicit denies

Public vs Private Subnets

Public Subnet

Internet-Facing Resources

- Route table includes route to Internet Gateway
- Resources can have public IP addresses
- Direct two-way internet connectivity
- Use for: Web servers, load balancers, bastion hosts
- Security: Must carefully configure security groups

Private Subnet

Internal Resources Only

- No direct route to Internet Gateway
- Resources use private IP addresses only
- Outbound internet via NAT Gateway
- Use for: Databases, application servers, internal services
- Security: Protected from direct internet access

Architecture Pattern: Deploy web tier in public subnets, application tier and database tier in private subnets. This creates defense in depth by limiting internet exposure.

VPC Endpoints: Private AWS Service Access

VPC endpoints enable private connections between your VPC and AWS services without requiring internet gateway, NAT device, VPN connection, or AWS Direct Connect. Traffic stays on the AWS network, improving security and reducing data transfer costs.



Gateway Endpoints

Route table-based connections for S3 and DynamoDB. Free to use. Add routes pointing to the endpoint in your route tables.



Interface Endpoints

ENI-based connections using AWS PrivateLink. Support most AWS services. Charged per hour plus data processing. Provide DNS names for private access.

Security Benefits: Keeps traffic off the public internet, reduces attack surface, enables tighter security group rules, supports VPC-specific access policies, simplifies compliance requirements for data residency.

EC2 Instance Security Fundamentals

01

Choose Secure AMIs

Use official AWS AMIs or trusted third-party images. Scan custom AMIs for vulnerabilities. Maintain golden image with security baselines.

02

Configure Security Groups

Restrict inbound rules to minimum required ports and source IPs. Avoid 0.0.0.0/0 except for web servers. Use separate security groups for different functions.

03

Use IAM Roles

Attach IAM roles to instances instead of embedding access keys in code. Roles provide temporary credentials that rotate automatically.

04

Enable Encryption

Encrypt EBS volumes using KMS keys. Enable encryption by default for new volumes. Use encrypted AMIs for sensitive workloads.

05

Patch Management

Use AWS Systems Manager Patch Manager for automated patching. Schedule regular maintenance windows. Track patch compliance across fleet.

S3 Bucket Security

S3 security requires multiple layers of defense. By default, S3 buckets and objects are private. However, misconfigurations can expose data publicly—one of the most common cloud security mistakes.

Block Public Access

Enable Block Public Access settings at bucket and account level. This prevents any public access regardless of bucket policies or ACLs. Should be enabled for all buckets unless public hosting is required.

Bucket Policies

JSON documents specifying who can access bucket and what actions they can perform. Use conditions to restrict by IP, VPC endpoint, or require encryption. Can grant cross-account access.

Access Control Lists

Legacy mechanism for granting basic permissions. AWS recommends using bucket policies and IAM policies instead. Disable ACLs when possible using S3 Object Ownership.

Encryption at Rest

Enable default encryption using SSE-S3 (AES-256) or SSE-KMS. Enforce encryption by denying unencrypted object uploads in bucket policy. Use SSE-KMS for audit trails and access control.

Versioning and MFA Delete

Enable versioning to protect against accidental deletion. Require MFA for permanent deletion of versions. Provides rollback capability and audit trail.

Application Layer Security

AWS WAF

Web Application Firewall protects web applications from common exploits like SQL injection and XSS. Deploy on CloudFront, ALB, API Gateway, or AppSync. Create rules to filter malicious traffic.

AWS Shield

DDoS protection service. Shield Standard automatically protects all AWS customers at no cost. Shield Advanced provides enhanced protection and 24/7 DDoS Response Team support for critical applications.

AWS Secrets Manager

Centrally manage secrets like database passwords and API keys. Automatically rotate credentials on schedule. Audit secret access with CloudTrail. Encrypt secrets using KMS.

Common Security Misconfigurations

Understanding common mistakes helps you avoid them. Here are real-world examples of misconfigurations and their consequences.

1

Public S3 Buckets

Mistake: Disabling Block Public Access or overly permissive bucket policies

Consequence: Sensitive data exposed to internet. Multiple breaches involving millions of records.

Prevention: Enable Block Public Access by default. Use Access Analyzer to detect public buckets.

2

Overly Permissive Security Groups

Mistake: Allowing 0.0.0.0/0 access on sensitive ports like SSH (22) or RDP (3389)

Consequence: Instances vulnerable to brute force attacks and unauthorized access.

Prevention: Restrict SSH/RDP to specific IP ranges. Use Systems Manager Session Manager for access.

3

Embedded Access Keys

Mistake: Hard-coding AWS access keys in application code or storing in version control

Consequence: Credentials exposed in GitHub or logs. Unauthorized resource access and cryptocurrency mining.

Prevention: Use IAM roles for applications. Store secrets in Secrets Manager. Scan code for credentials.



Encryption & Monitoring

Data Encryption at Rest

Encryption protects data if physical media is compromised. AWS provides multiple encryption options across services, all using industry-standard AES-256 encryption.

AWS Key Management Service (KMS)

Managed service for creating and controlling encryption keys. Supports automatic key rotation. Integrates with most AWS services. Provides detailed audit logs via CloudTrail. Use for centralized key management and compliance requirements.

Server-Side Encryption (SSE)

SSE-S3: AWS-managed keys. Easiest option, no key management required.

SSE-KMS: Keys managed in KMS. Provides audit trail and access controls.

SSE-C: Customer-provided keys. Customer manages keys outside AWS.

EBS Volume Encryption

Encrypt volumes using KMS keys. Encryption happens at the hypervisor level—transparent to applications. Snapshots of encrypted volumes are automatically encrypted. Enable encryption by default setting for account.

RDS Encryption

Encrypt database instances and snapshots using KMS. Must enable at creation time—cannot encrypt existing unencrypted instance. Automated backups, read replicas, and snapshots inherit encryption settings.

Data Encryption in Transit

TLS/HTTPS Everywhere

Transport Layer Security (TLS) encrypts data moving between clients and servers. Always use HTTPS for web traffic, APIs, and service communications.

AWS Certificate Manager (ACM): Provision, manage, and deploy SSL/TLS certificates at no cost. Automatic renewal prevents certificate expiration. Integrates with CloudFront, ALB, and API Gateway.

Implementation:

- Load balancers: Configure HTTPS listeners with ACM certificates
- CloudFront: Require HTTPS between viewers and CloudFront
- API Gateway: Enable TLS for custom domains



VPN and Direct Connect

AWS Site-to-Site VPN: Encrypted connections between your network and AWS. Uses IPsec tunnels for secure communication.

AWS Direct Connect: Dedicated network connection from on-premises to AWS. Can be combined with VPN for encryption.

AWS PrivateLink: Private connectivity between VPCs and AWS services without traversing public internet.

Logging and Monitoring Tools



AWS CloudTrail

Records all API calls made in your account. Essential for security analysis, compliance auditing, and troubleshooting. Enable for all regions. Store logs in encrypted S3 bucket with MFA Delete enabled.



AWS Config

Continuously assesses resource configurations against desired settings. Track configuration changes over time. Create Config Rules to automatically detect non-compliant resources. Integrates with Systems Manager for remediation.



Amazon Inspector

Automated security assessment service. Scans EC2 instances and container images for vulnerabilities and deviations from best practices. Prioritizes findings by severity. Continuous scanning for new vulnerabilities.



Amazon CloudWatch

Collects and tracks metrics, logs, and events. Set alarms on security-relevant metrics. Create dashboards for visibility. Use CloudWatch Logs Insights for log analysis and anomaly detection.



Amazon GuardDuty

Intelligent threat detection using machine learning. Analyzes CloudTrail, VPC Flow Logs, and DNS logs. Detects compromised instances, reconnaissance activity, and unauthorized access. Provides detailed findings with severity levels.



AWS Security Hub

Central security and compliance dashboard. Aggregates findings from GuardDuty, Inspector, Macie, and other services. Runs automated security checks against AWS best practices. Provides compliance reports for standards like CIS.

Incident Response Workflow

Effective incident response minimizes impact and recovery time. AWS provides tools to support each phase of the incident response lifecycle.

1

Detection

GuardDuty identifies suspicious activity. CloudWatch alarms trigger on anomalies. Security Hub aggregates findings across services. Automated notifications via SNS or EventBridge.

2

Analysis

Review CloudTrail logs to understand scope. Check VPC Flow Logs for network activity. Analyze affected resources using Config timeline. Determine root cause and impact assessment.

3

Containment

Isolate compromised instances with security group changes. Revoke IAM credentials. Take snapshots before changes. Prevent lateral movement. Document all actions.

4

Eradication

Remove malware and backdoors. Patch vulnerabilities. Terminate compromised instances. Restore from clean backups. Update security controls to prevent recurrence.

5

Recovery

Restore services from secure backups. Monitor closely for signs of persistence. Validate fixes are effective. Return to normal operations. Update runbooks and procedures.

6

Post-Incident

Document timeline and actions taken. Identify lessons learned. Update incident response plans. Implement preventive controls. Train team on new procedures.

Sample Incident Response Scenario

Scenario: GuardDuty detects unusual API calls from an EC2 instance suggesting potential compromise.



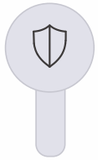
Alert Triggered

GuardDuty finding: UnauthorizedAccess:EC2/SSHBruteForce. Security team notified via SNS.



Investigation

Review CloudTrail: Confirmed suspicious API calls. VPC Flow Logs show connections to unusual IPs. Instance accessed S3 buckets it shouldn't access.



Containment

Update security group to deny all traffic. Snapshot EBS volume for forensics. Revoke instance IAM role. Block malicious IPs in NACL.



Eradication

Terminate compromised instance. Launch replacement from secure AMI. Update SSH keys. Patch vulnerability that allowed initial access.



Validation

Monitor new instance for 48 hours. Verify no unauthorized access. Confirm security controls effective. S3 access properly restricted.

Key Takeaways

Shared Responsibility Is Foundational

AWS secures the infrastructure; you secure your applications, data, and access controls. Understanding this division is critical for compliance and security posture. Responsibility increases as services become less managed (IaaS → PaaS → SaaS).

IAM and Least Privilege Are Non-Negotiable

Identity management is your first line of defense. Implement least privilege, use temporary credentials, enable MFA everywhere, and regularly audit permissions. 80% of breaches involve compromised credentials—get IAM right.

Defense in Depth Requires Multiple Layers

No single control is sufficient. Combine network controls (security groups, NACLs), encryption, monitoring, and access controls. Each layer provides another opportunity to detect and block threats.

Monitoring + Automation = Strong Security

Enable comprehensive logging with CloudTrail and CloudWatch. Deploy automated threat detection with GuardDuty. Automate responses to common scenarios. You can't protect what you can't see—visibility is everything.

Security Is Continuous Improvement

Threats evolve constantly. Regularly review and update security controls. Conduct security assessments. Learn from incidents. Test your incident response plans. Security is a journey, not a destination.

What is Automated Security?

Automated security uses cloud-native tools, policies, and workflows to continuously monitor, detect, and respond to security threats with minimal human intervention.



Why Automation Matters

- Faster threat detection
- Reduced response time
- Improved compliance
- Reduced human error
- Scalable security management

Key Benefits

- Continuous monitoring
- Real-time alerts
- Automated remediation
- Centralized governance

What is AWS Organizations?

AWS Organizations is a service that helps centrally manage and govern multiple AWS accounts from a single location.

Key Features

- Centralized account management
- Policy-based governance
- Consolidated billing
- Security standardization

Organizational Structure

- Root
- Organizational Units (OUs)
- AWS Accounts

Example

- A company can separate:
- Production Account
- Development Account
- Testing Account
- Security Account

This reduces risk and improves access control.

Threat Detection in AWS

- AWS provides integrated security services that continuously monitor cloud environments for suspicious activities.

❖ **Core Services**

Amazon Guard Duty

- Threat detection service
- Detects malicious activity and unauthorized behaviour

Enhancing Multi-Account Security

➢ **Service Control Policies (SCPs)**

SCPs define maximum permissions for accounts within an organization.

➢ **Centralized Security Controls**

- Consistent security policies
- Standardized configurations
- Central auditing

➢ **Benefits**

- Reduced attack surface
- Better compliance management
- Stronger governance
- Isolation of workloads

➢ **Best Practice**

- Use a dedicated Security Account to monitor and manage security services across all accounts.

AWS Security Hub

- Centralized security dashboard
- Aggregates findings from multiple services

Amazon Inspector

- Vulnerability assessment
- Continuous security scanning

AWS Config

- Resource compliance monitoring
- Configuration tracking

Incident Response Framework

- AWS security services work together to support incident handling.

Workflow

- Monitor environment
- Detect suspicious activity
- Generate security findings
- Notify security teams
- Initiate remediation
- Recover and analyse

Supporting Services

- GuardDuty
- Security Hub
- CloudTrail
- CloudWatch
- AWS Lambda

Goal: Minimize the impact of security incidents through rapid response.

Automated Security Notifications

- Timely alerts help security teams respond quickly to threats.

Notification Components

Amazon EventBridge

- Event-driven architecture
- Routes security events automatically

Amazon SNS (Simple Notification Service)

- Sends notifications
- Email alerts
- SMS alerts
- Application notifications

Example

GuardDuty Finding → EventBridge Rule → SNS Notification → Security Team Alert

What is Automated Remediation?

Automated remediation automatically executes predefined actions when a threat is detected.

Common Remediation Actions

Isolate a Compromised EC2 Instance

- Remove from network
- Restrict access

Block Malicious IP Addresses

- Update Security Groups
- Modify WAF rules

Disable Compromised Credentials

- Revoke IAM access keys
- Force password reset

AWS Services Used

- AWS Lambda
- Systems Manager Automation
- EventBridge
- Security Hub

Case Study – AWS Threat Detection in Action

Scenario

- A company hosts customer applications on AWS.
- An attacker obtains stolen credentials and attempts to access cloud resources from an unusual geographic location.

Potential Risks

- Unauthorized access
- Data theft
- Resource misuse
- Compliance violations

Threat Detection and Response Process

Step 1: Threat Detection

Amazon GuardDuty Detects:
- Unusual login activity
- Access from suspicious IP addresses
- Abnormal API calls

Step 2: Security Finding Generated

GuardDuty sends findings to:
- AWS Security Hub
- Amazon EventBridge

Step 3: Alert Notification
Security team receives:
- Email alerts
- Dashboard notifications
- Incident tickets

Response Actions Triggered

AWS Lambda Automatically:
- Disables compromised IAM credentials
- Tags affected resources
- Isolates impacted EC2 instances
- Creates incident records

Security Team Activities
- Investigate logs
- Assess impact
- Verify containment
- Restore normal operations

Outcome
- Threat contained within minutes, reducing risk and preventing further compromise.

Discussion Questions

Q1: A developer wants to give their application access to S3. Should they create an IAM user with access keys or use an IAM role? Why?

Q2: Your company stores customer data in S3. What combination of security controls would you implement to ensure this data is protected?

Q3: GuardDuty alerts you to suspicious API activity from an IAM user. Walk through your incident response steps.

Q4: You need to allow a partner company to access specific S3 objects in your account. What's the most secure way to grant this access?

Q5: Describe the security controls you would implement for a three-tier web application (web, app, database layers) deployed in AWS.



Additional Resources and Next Steps

AWS Documentation

- AWS Security Best Practices Whitepaper
- AWS Well-Architected Framework - Security Pillar
- IAM Best Practices Guide
- VPC Security Documentation
- AWS Security Blog

Hands-On Learning

- AWS Security Workshops
- AWS Skill Builder Security Learning Plans
- AWS Security Hub with CIS Benchmarks
- CloudGoat (Intentionally vulnerable AWS environment)

Certifications

AWS Certified Security - Specialty

Validates expertise in securing AWS workloads. Covers incident response, logging, monitoring, infrastructure security, and identity management.

What's Next?

1. Complete hands-on labs in AWS console
2. Set up a practice account with security controls
3. Review IAM policies for sample applications
4. Simulate an incident response scenario
5. Take the AWS Security Fundamentals course